02 本体、知识图谱、语义层与 RAG，到底是什么关系?

原创 陈乔 通往AI之桥 2026年10月26日 12:30 北京

从概念模型到 AI 工程架构，一次讲清四个经常被混用的技术概念

通往AI之桥 | 本体论在 AI 工程中的实践系列 · 02

做 AI 项目时，我们经常会听到这些词：

本体（Ontology）、知识图谱（Knowledge Graph）、语义层（Semantic Layer）、RAG（检索增强生成）。

有的项目在做知识图谱，有的项目在搭建企业语义层，有的项目给大模型接入 RAG，还有一些厂商开始强调 Ontology 是企业 AI 的核心基础。

它们看起来都在解决知识和语义问题，于是很容易产生一个疑问：

这四种技术，到底有什么区别？它们是相互替代的关系，还是应该组合起来使用?

如果把本体、知识图谱、语义层和 RAG 当成四种可以互相替代的技术，很容易在架构设计阶段就走错方向。

我更倾向于用一句话来理解它们：

> **本体定义业务世界的含义，知识图谱组织业务世界中的事实，语义层把业务含义映射到数据和指标，RAG 则为大模型检索并组织回答问题所需要的证据。**

四者可以相互配合，但并不是每个项目都必须同时具备四者。

这篇文章，我们就从一个具体的业务问题开始，把它们逐一拆开。

一、先看一个真实的工程问题

假设我们正在建设一个企业级 AI 助手。

业务人员提出问题：

> “为什么客户 A 的订单延迟了？这次延迟是否会影响客户交付承诺？如果问题来自某个软件组件，哪些客户还会受到影响？”

一个普通的大模型可能会尝试回答，也可以通过 RAG 检索订单文档、物流记录、故障报告和软件发布说明。

但要可靠地回答这个问题，系统至少需要完成几件事：

1. 确认“客户 A”指的是哪个客户实体，而不是名称相似的其他客户。
2. 找到客户的订单，识别对应的物流单和交付承诺。
3. 依据企业认可的定义判断什么是“延迟”。
4. 找出订单延迟与哪个系统、服务或软件组件存在关系。
5. 检索相关故障记录、技术文档和软件版本信息。
6. 如果需要进一步处理，再判断什么操作可以执行、由谁审批。

你会发现，这不是一个单纯的文本问答问题。

它同时涉及业务概念、真实事实、数据定义、证据检索和操作规则。

这正是四种技术开始产生分工的地方。

二、本体：先定义我们究竟在谈论什么

本体要解决的第一个问题是：**业务世界中的对象是什么，它们具有哪些属性，彼此之间存在什么关系，又有哪些规则需要遵守？**

仍然以电商业务为例，我们可以定义以下概念：

客户（Customer）

订单（Order）

商品（Product）

物流单（Shipment）

软件服务（Service）

软件组件（Component）

软件版本（SoftwareVersion）

然后定义它们的关系：

客户创建订单。

订单包含商品。

订单对应物流单。

订单处理依赖某个软件服务。

软件服务依赖一个或多个软件组件。

某个软件版本可能影响特定的软件服务。

我们还可以进一步定义属性、约束和业务规则，例如：订单承诺交付时间、物流状态、服务依赖关系，以及什么条件下订单才算延迟。

这组定义就是领域本体的一个简化示例。

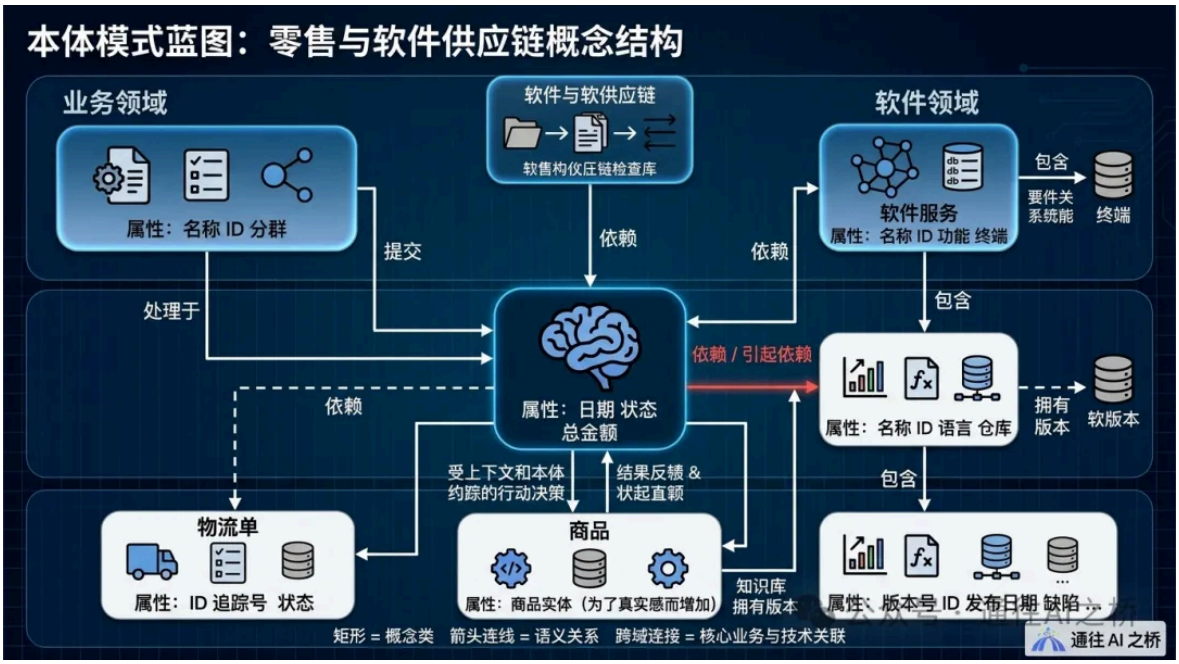
本体不只是把名词列出来，也不只是画一张关系图。它还需要回答：两个名词是不是同一个概念？两种关系是否具有不同含义？哪些属性属于哪个实体？什么规则必须成立？

在更正式的知识工程中，本体可以使用 OWL 等形式化语言表达，并支持机器处理的语义定义和逻辑推断。W3C 的 OWL 标准对相关机制进行了系统介绍：OWL 2 Web Ontology Language Primer。

本体关注的是业务世界的概念结构，而不是某个数据库具体怎样存储数据。

例如，CRM 系统可能用 customer_id 表示客户，订单系统可能用 buyer_no 表示购买方，财务系统又可能使用 account_code 表示结算主体。本体可以在业务层明确这些概念的含义及其关系，再通过数据映射将它们连接起来。

这也是本体在 AI 工程中最重要的价值之一：它可以成为多个系统、多个 Agent 共同遵循的业务语义契约。



【图 1：本体示意图】

三、知识图谱：把业务世界中的事实组织起来

有了本体定义的概念和关系，下一步就是把实际业务数据组织起来。

例如：

客户 A 创建了订单 O1001。

订单 O1001 对应物流单 S2001。

订单 O1001 依赖服务 PaymentService。

PaymentService 使用组件 KafkaClient。

KafkaClient 的某个版本存在已知缺陷。

某个客户环境当前正在运行这个版本。

这些是具体的业务事实，而不再只是概念定义。

我们可以把它们组织成一个图：

客户 A → 订单 O1001 → 服务 PaymentService → 组件 KafkaClient → 软件版本 V3.6

每个节点代表实体，每条边代表实体之间的关系。知识图谱可以进一步保存节点和边的属性、数据来源、时间信息和可信度。

在 RDF 这样的标准数据模型中，事实可以表示为“主体—谓词—客体”三元组。例如：

订单O1001 — 依赖服务 — PaymentService

W3C 的 RDF 规范使用这种三元组结构表达资源及其关系，并将一组三元组组织成 RDF 图：RDF 1.1 Concepts and Abstract Syntax。

需要特别注意：

知识图谱不等于本体。

本体规定我们有哪些概念、概念如何定义，以及关系具有怎样的含义；知识图谱则组织具体的实体、关系和事实。

可以把它们理解成两种不同层次的东西：

本体定义“订单”“客户”“依赖”等概念以及允许的关系。

知识图谱记录现实中有哪些订单、客户和依赖关系。

知识图谱可以依据本体构建，也可以在更灵活的 Schema 或较弱的语义约束下建立，并不一定要拥有完整、严格的形式化本体。

另外，知识图谱也不意味着所有数据都必须从原始系统迁移到图数据库。工程上可以只存储实体标识与关系索引，将详细数据保留在原有业务系统中，再按需访问。



【图 2：业务关系网络——影响分析路径】

四、语义层：让不同系统对业务指标有共同的理解

接下来，考虑另一个问题。

业务负责人问：

> “本月订单延迟率是多少？哪些客户受到的影响最大？”

这时即使已经有了本体和知识图谱，我们仍然需要回答一个很实际的问题：**“订单延迟率”到底怎样计算?**

一个系统可能把订单状态为 DELIVERED_LATE 的订单算作延迟；另一个系统可能将超过承诺交付时间的订单全部算作延迟；还有一个报表可能排除了客户主动变更交付时间

的订单。

这些定义都可能在技术上成立，却不能保证业务口径一致。

这就是语义层（Semantic Layer）的价值所在。

语义层通常将业务概念、数据字段、计算指标、维度、过滤条件和底层数据模型联系起来，让不同应用能使用一致的业务术语和计算口径。

例如，语义层可以定义：

“订单”映射到订单数据集及其唯一标识。

“客户”映射到经过统一识别的客户实体。

“承诺交付时间”对应哪个数据字段，以及其时间和时区规则。

“订单延迟率”采用什么分子、分母、时间窗口与排除条件。

“受影响客户数”按照客户、订单还是客户环境去重。

当管理驾驶舱、分析系统和 AI Agent 都使用同一个经过治理的指标定义时，就不必让每个应用各自编写并解释一遍计算逻辑。

Microsoft Fabric IQ 是一个值得关注的例子。微软将语义模型和 Ontology 作为不同但可以协作的企业业务上下文层：语义模型侧重可信指标、维度和分析定义；Ontology 则进一步描述业务实体、关系、规则和数据绑定，并让 Agent 使用共享业务语义。Microsoft Fabric IQ 文档。

这条路线也提醒我们：业务语义应尽可能复用已验证的指标定义，而不应让每个 AI 应用各自解释指标。

所以，语义层与本体有交集，却不完全相同：

语义层更关注“业务语言怎样映射到可查询的数据和指标”，本体更关注“业务世界中的概念及其关系与约束是什么”。

某些平台会将二者整合在一起；另一些平台则会分别维护，再通过映射进行连接。

五、RAG：为大模型找到回答问题所需要的证据

前三种技术主要关注业务世界的描述、事实组织和数据含义。RAG 解决的是另一类问题：

当大模型需要回答一个问题时，怎样从外部知识中找到相关证据，并让模型基于这些证据生成答案？

RAG，即 Retrieval–Augmented Generation，通常译为“检索增强生成”。

2020 年，Lewis 等人在 NeurIPS 发表了《Retrieval–Augmented Generation for Knowledge–Intensive NLP Tasks》，提出将预训练生成模型与外部的非参数化知识存储结合起来，用于知识密集型任务。论文原文。

一个常见的 RAG 流程包含：

1. 文档解析与切分。
2. 为文本片段建立向量索引或其他检索索引。
3. 根据用户问题检索相关内容。
4. 对检索结果进行过滤、排序和上下文组装。
5. 将问题与证据交给大模型生成答案。
6. 在需要时返回来源引用、证据或可核验的数据。

在上面的订单案例中，RAG 可以检索物流操作记录、故障报告、Kafka 版本说明和已发布的技术公告，帮助模型解释为什么某个服务出现问题。

但传统的向量检索有一个局限：它主要依据文本或向量相似度找出相关片段，并不天然知道业务实体之间的完整关系。

例如，“订单 O1001 受 Kafka 版本缺陷影响”这条关系，可能散落在几份不同的文档中。只依赖普通文本相似度检索，不一定能稳定地找到完整的依赖路径。

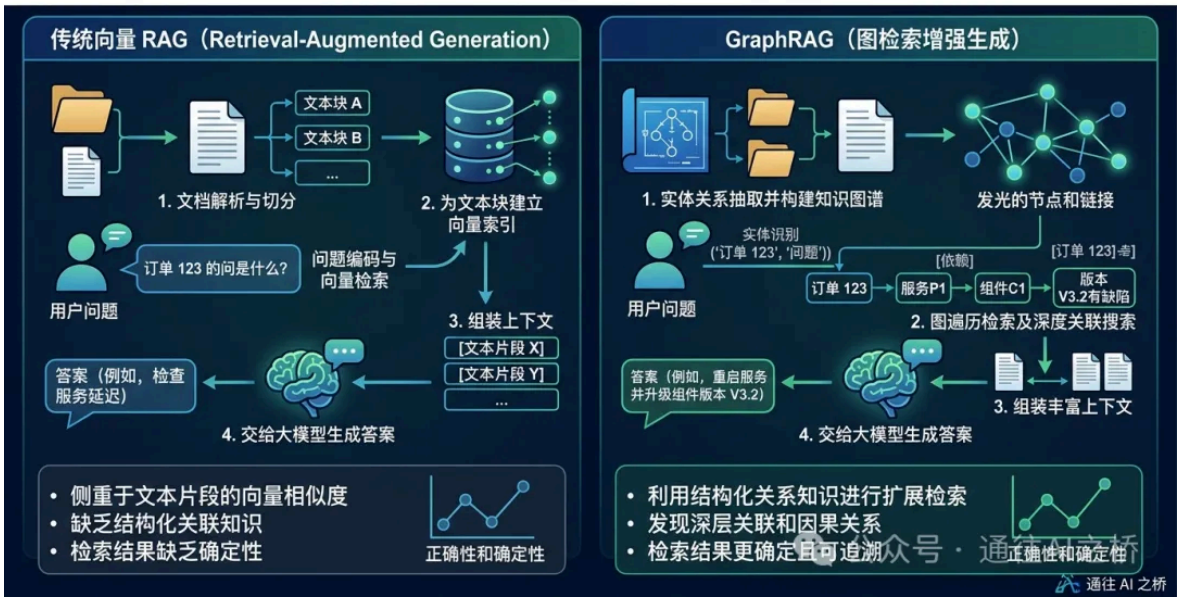
这时就可以考虑将知识图谱与 RAG 结合，形成 GraphRAG。

Microsoft Research 将 GraphRAG 描述为结合文本信息抽取、图结构分析、LLM 提示和摘要的检索增强生成技术：Microsoft Research GraphRAG。

Neo4j 的 GraphRAG 文档也介绍了使用向量检索、全文检索与图遍历相结合的方法，从相关文本片段扩展到有关联的实体、关系和其他事实：Neo4j GraphRAG。

因此，**RAG 是一种检索与生成架构，不是某一种固定的数据存储技术；知识图谱可以成为 RAG 的检索来源之一，但不是所有 RAG 都需要知识图谱。**

图3：RAG 与 GraphRAG 对比示例



【图 3：RAG 与 GraphRAG 对比】

六、四者究竟是什么关系？一张表看清楚

维度	本体 Ontology	知识图谱 Knowledge Graph	语义层 Semantic Layer	RAG
核心问题	业务概念和关系如何定义?	现实中的对象和事实如何连接?	业务术语和指标怎样映射到数据?	如何检索证据并生成答案?
主要对象	类、属性、关系、规则与约束	实体、关系、属性和事实	指标、维度、数据映射与计算口径	文档片段、检索结果、上下文
主要作用	提供共享语义和显式约束	组织事实与关系，支持关联查询	统一业务口径和数据访问	为模型提供外部证据
典型实现	本体定义、Schema、元数据等	图数据库、RDF Store 或关系表等	指标定义、语义模型、数据目录和映射	向量索引、全文索引、文档库等

维度	本体 Ontology	知识图谱 Knowledge Graph	语义层 Semantic Layer	RAG
是否必须使用图数据库	不必须	不一定，但常见	不必须	不必须
是否直接负责生成答案	通常不负责	通常不负责	通常不负责	在完整 RAG 流程中与模型配合生成答案
主要局限	建模成本和语义维护成本	数据质量、关系完整性与更新问题	跨系统口径治理及数据映射复杂性	检索遗漏、上下文噪声和生成错误

这张表也揭示了一个关键事实：四种技术并不是四个互相争夺同一位置的产品，而是可以从不同角度构成一套企业 AI 知识架构。

不过，它们之间也并非严格的单向依赖关系。语义层可以独立存在，知识图谱也可以先于本体建立；RAG 可以只检索文档。本体可以帮助统一概念、约束和数据映射，却不意味着所有实体、指标和文档都必须以同一种形式存储。

七、如果将四者组合起来，一套 AI 工程架构会是什么样？

回到最初的问题：找出订单为什么延迟，以及哪些客户会受到影响。

我们可以采用以下组合方式：

第一步：本体定义业务世界。

明确客户、订单、物流、软件服务、组件与版本等概念，并定义允许的关系和重要业务约束。

第二步：知识图谱连接真实事实。

把实际订单、客户、服务、组件和软件版本连接起来，并保留来源与更新时间。若企业不需要复杂图查询，也可以通过关系数据库和索引完成部分工作。

第三步：语义层统一业务指标。

明确什么是订单延迟、受影响客户、交付承诺以及故障率，绑定到底层真实数据和计算逻辑。

第四步：RAG 检索非结构化证据。

查找故障报告、运行日志、版本发布说明和处理手册，提供与问题相关的证据。

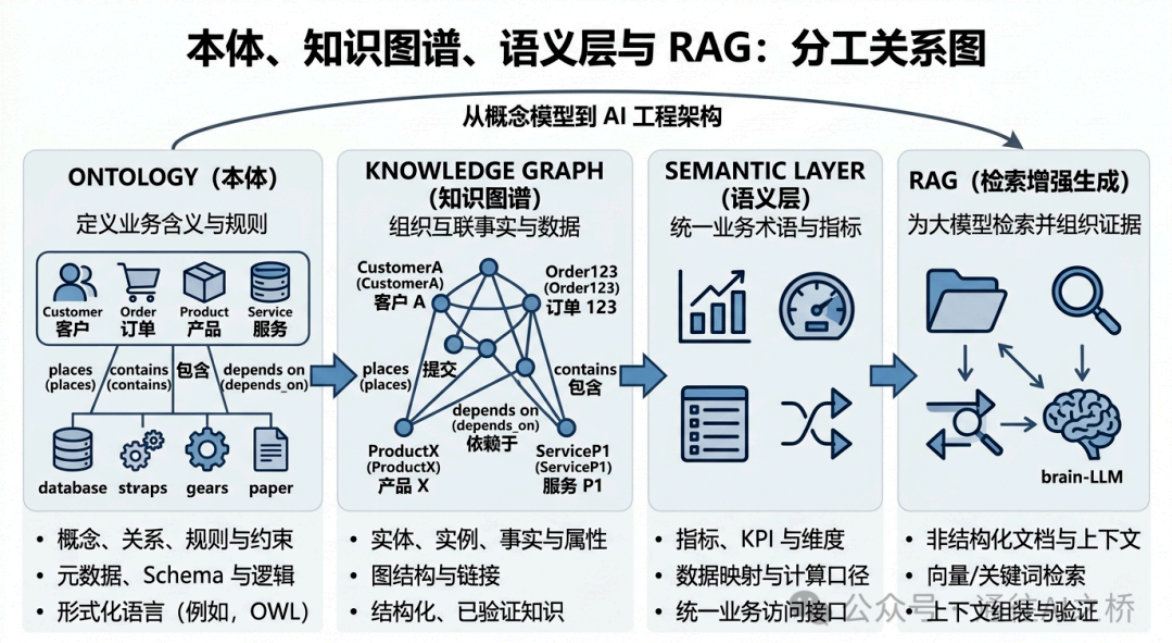
第五步：AI Agent 组合结果并执行受控操作。

Agent 先依据业务定义理解问题，再查询事实和检索证据，最后给出可追溯的解释。如果需要重启服务、升级组件或通知客户，则调用受控工具，并遵循授权、审批、测试和审计流程。

这套设计可以概括成：

> **共享业务定义 + 可追踪的事实关系 + 统一的数据口径 + 外部证据检索 + 受治理的工具执行。**

它比单纯给大模型接入一个知识库更加完整，但具体项目不必一次性建设全部层次。应根据业务关系的复杂程度、数据口径的分歧、风险以及维护成本选择组合。



【图 4：端到端架构图】

八、在 AI 软件工厂中，四者可以怎样协同?

如果把这套思路用于 AI 软件工厂，四种技术的价值会更加明显。

软件工厂会持续处理需求、组件、版本、构建、测试、安全漏洞、发布、部署和运维事件。如果它们只存在于不同的代码仓库、CI/CD 平台、制品库和运维系统中，AI 就需要不断重新理解它们的名称、依赖和含义。

我们可以建立一组统一的业务概念：

Requirement：需求。

Product：软件产品。

Component：软件组件。

Version：版本。

Build：构建。

TestResult：测试结果。

Finding：安全或质量问题。

Release：发布。

Deployment：部署。

Environment：运行环境。

本体负责定义这些概念和关键关系；知识图谱或其他关系存储负责连接实际项目中的实体和依赖；语义层负责统一产品质量、版本覆盖、漏洞影响范围等指标；RAG 则从设计文档、发布记录、故障报告和技术手册中提取证据。

当一个高危漏洞被发现时，AI 就有机会沿着“漏洞—组件版本—软件产品—客户部署环境”的关系找出受影响对象，再结合修复手册和兼容性说明形成升级方案。

这并不意味着本体可以代替 CI/CD 或安全门禁。真正的构建、测试、漏洞确认、发布授权和生产部署，仍然必须交给可靠的工程工具和治理机制执行。

本体的作用是让 Agent 更清楚自己正在处理什么对象、这些对象有什么关系、哪些约束必须满足，以及它的下一步行动会影响什么。

九、工程实践：什么时候值得用四者组合？

我建议从问题出发，而不是从技术名词出发。

如果只是企业文档问答，文档切分、混合检索、RAG 评估和来源引用可能已经够用。此时强行建立完整本体和知识图谱，未必有足够收益。

如果企业的问题是“同一个指标在不同系统里算得不一样”，那么首先应该解决语义层、指标口径和数据映射问题。

如果问题依赖大量实体之间的关系，例如软件依赖分析、供应链风险、复杂故障传播或客户影响范围，那么知识图谱或其他显式关系模型可能更有价值。

如果多个 Agent 要在同一个业务领域里共享概念，并且开始执行会影响真实业务状态的操作，那么本体、业务规则、权限控制与审计机制就更值得考虑。

一个务实的落地顺序是：

1. 选择一个高价值业务问题，明确答案或操作应满足什么标准。
2. 先统一核心概念和指标口径。
3. 确认现有数据源、文档和系统之间需要哪些关系。
4. 根据查询需求选择关系数据库、图数据库、向量检索或混合存储。
5. 为 RAG 和 Agent 设计可验证的测试集，检查检索召回、答案正确性、引用有效性和任务完成率。
6. 在需要执行操作时，再加入权限、审批、状态约束和审计闭环。

在这个过程中，本体不必一次性覆盖整个企业。先建立一个范围清楚、能被实际应用验证的领域模型，往往比一开始追求一个宏大的“企业统一本体”更合理。

十、最后总结：不要把四种技术混为一谈，也不要把它们割裂开来

回到标题中的问题：

本体、知识图谱、语义层和 RAG，到底是什么关系？

我的理解是：

> **本体定义业务世界的含义，知识图谱组织具体的实体和关系，语义层让业务指标和数据映射保持一致，RAG 则把外部证据带入模型的生成过程。**

它们在架构上相互补充，但不必绑定成固定的四件套。真正优秀的 AI 工程设计，不是为了使用更多技术而增加系统复杂度，而是根据业务问题选择合适的语义模型、事实存储、数据访问和检索机制。

大模型让我们能够更灵活地理解语言、生成方案和调用工具。但在复杂的企业环境里，单靠语言能力并不足以保证行动正确。

我们还需要清晰的概念、可靠的事实、一致的数据口径、可追溯的证据，以及能够约束操作的工程规则。

这也正是我持续关注本体论在 AI 工程中应用的原因。

> **本体负责说明业务世界是什么，数据与知识负责描述它现在是什么样，RAG 负责找到支持判断的证据，而 Agent 负责在约束内采取行动。**

当这些能力结合起来，AI 才更有机会从“知道一些信息”，走向“理解业务、可靠执行，并能够接受验证”。

系列预告

这是「通往AI之桥｜本体论在 AI 工程中的实践」系列第二篇。

下一篇计划讨论：

《怎样从一个真实业务场景开始设计本体？》

我们将以一个具体业务为例，从核心实体、属性、关系、规则和数据映射开始，展示怎样将本体设计转化为可验证的工程资产，而不是停留在概念图和术语表上。

参考资料

说明：以下资料用于核对技术概念、标准和产品定位。厂商介绍反映公开产品表述，不等同于独立第三方效果评测。

W3C — OWL 2 Web Ontology Language Primer

W3C — RDF 1.1 Concepts and Abstract Syntax

Lewis et al. — Retrieval–Augmented Generation for Knowledge–Intensive NLP Tasks, NeurIPS 2020

Microsoft Research — GraphRAG

Neo4j — GraphRAG

Palantir Foundry — Ontology

Microsoft Fabric IQ — Overview

本体论在AI工程中的实践 · 目录 ≡

〈上一篇 · 01 AI项目为什么需要本体？–本体论在 AI 工程中的实践系列

作者提示: 内容由AI生成